Graphs

- Cycle Detection
- Topological Sorting
- Dijkstra's algo for SSSP in weighted graph
  = BFS — unweighted graph

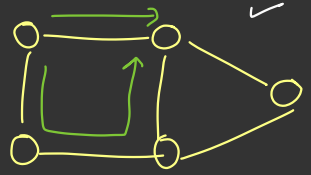
→ No named algorithm
→ Graphs: ✓

Cycle Detection

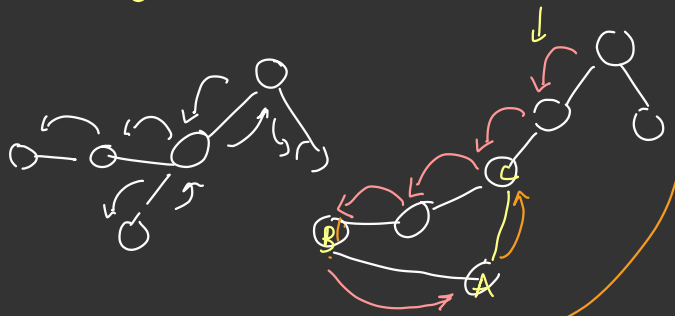
② check if a given graph is a Tree

- Undirected
  - BFS
  - DFS (todo)
- directed

Idea - if you are visiting a node more than once, it contains cycle



DFS



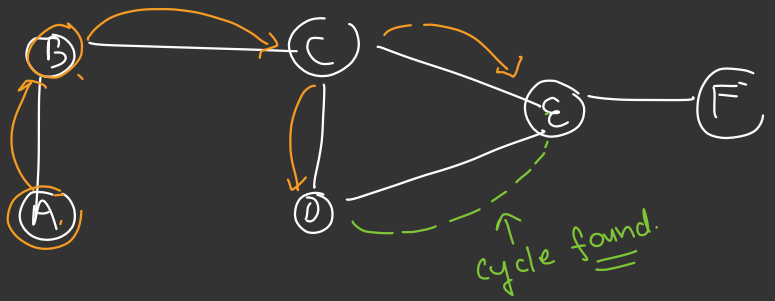
nbr should not be same as parent

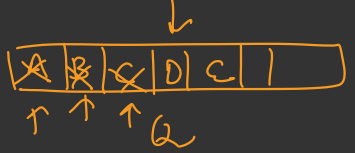
A → (B, C)

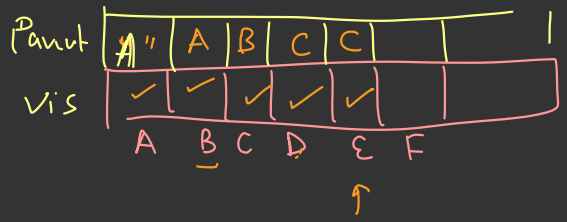
already visited

hence cycle is present

BFS Algo → same as DFS, maintain parent & check if a node is already visited







parent[c] == nbr
↳ skip

A → {B}
B → {A, C}
↑ skip

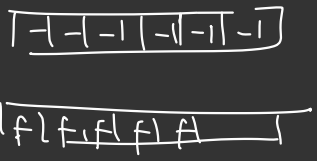
D → {Q, E}
↑ ↑
skip

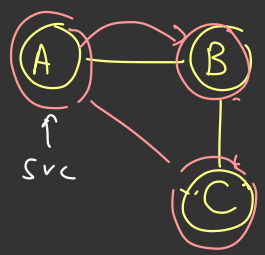
C → {B D E}
↑
nbr

BFS

parent = init()

visited = init()



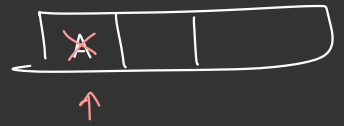


queue q

q push (src)

visited [src] = True

parent [src] = src



while (!q.empty()) {

f = q.poll()

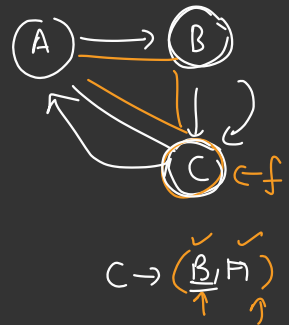
for (nbr : adjlist[f]) {

// not visited

if (!visited[nbr]) {

visited[nbr] = True

q.push(nbr)

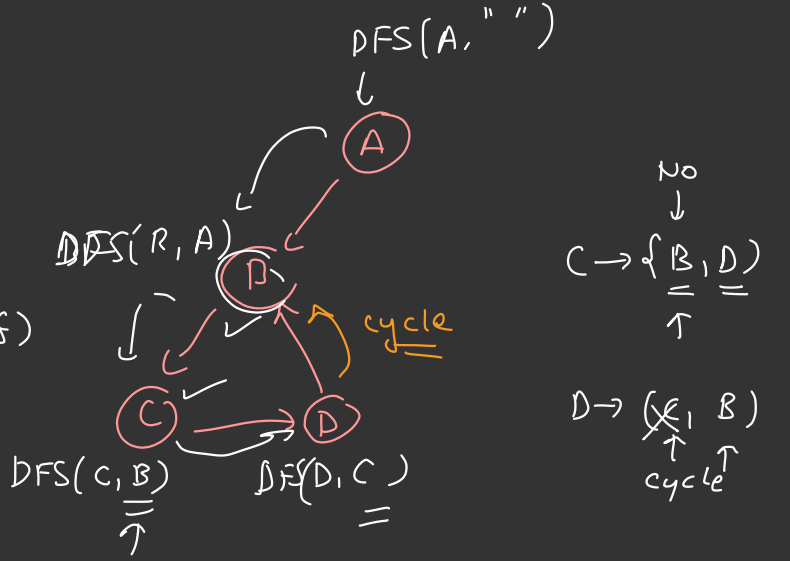


nbr should not be parent of f

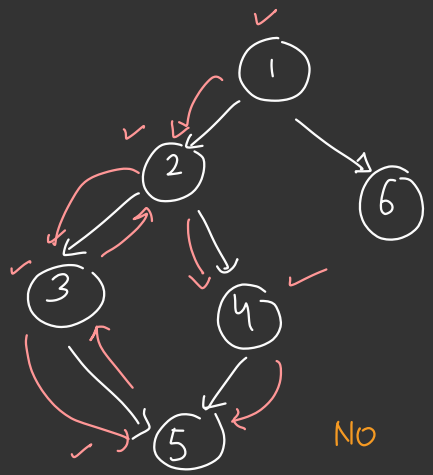
```
        parent[nbr] = f                                   nbr  nbr
    }
    else if ( visited[nbr] == True && nbr != parent[f] ){
            return True;
    }
}
}
return false
```

DFS = TODO [HW]

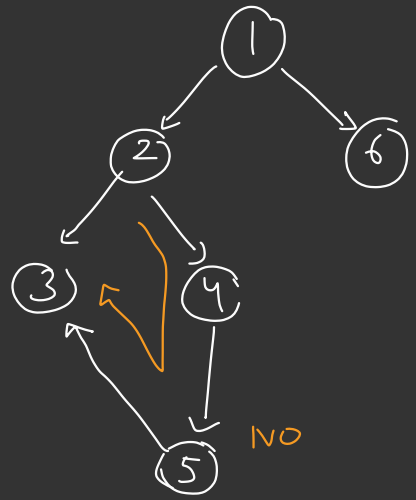
(just pass the parent of cNode in the fn call itself)



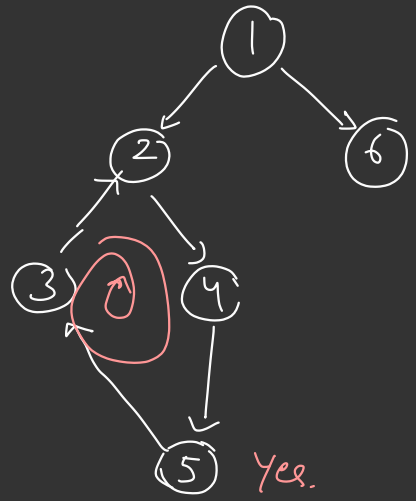
Cycle Detection in a Directed Graph



NO



NO



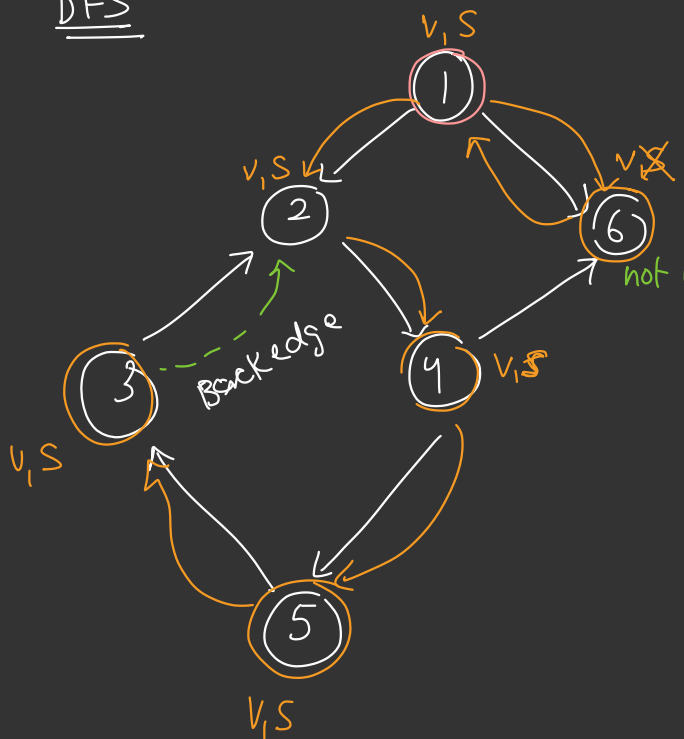
Yes.

DFS fails

↓

modify the algo

DFS



not in stack, && visited
↓
6 is not in the path 4

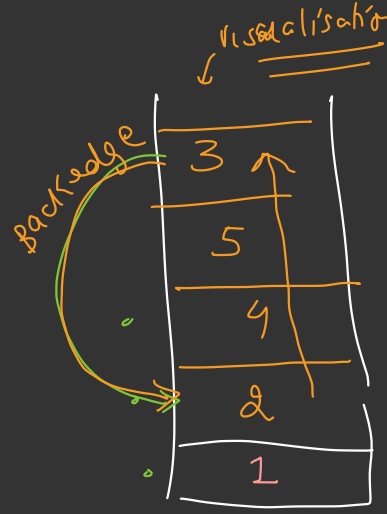
adj list (Dir Graph)

1 → (6, 2)
2 → 4
3 → 2
4 → 6, 5
5 → 3
6 → " "

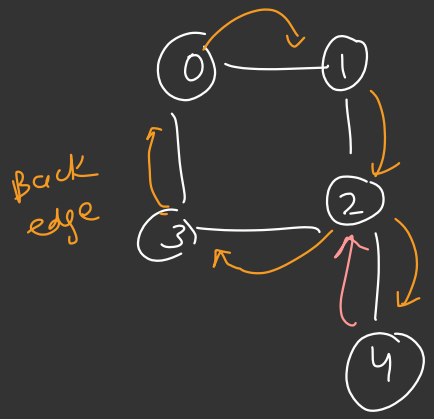
2 is in the path of 3 — and is already visited

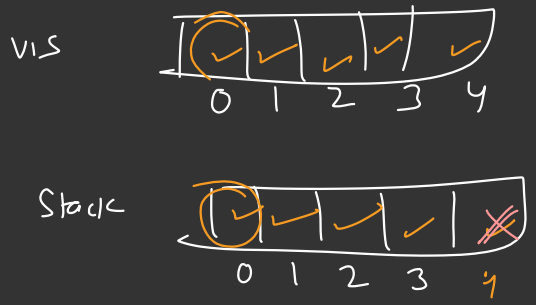
Backedge x → y ⇒ both visited && in stack
↓
Backedge → cycle

Call Stack



↓
maintain stack_array separately





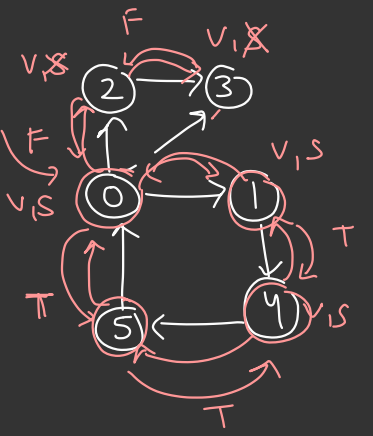

```
bool dfs( adjlist, N ) {
    visited = init()
    stack_arr = init()
    return dfsHelper( adjlist, 0, vis, stack_arr )
}
```

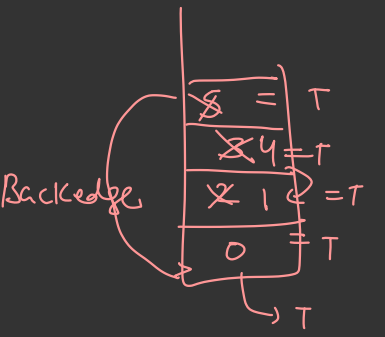
F	F	-	-	-

F	F	-	-	-

3

```
dfsHelper ( adjList , node , vis , stack_arr ) {

Base Case    if ( visited[node] == T && stack_arr[node] == T )
                 return True
             if( visited[node] == T && stack_arr[node] == F ){
                 return false

Rec Case     3

  - visited[node] = T
  - stack_arr[node] = T

     for( nbr : adjList[node] ){
          if( dfsHelper (adjList, nbr, vis, stack_arr) ){
               return True
          }
     }
}
```





- stack_arr[node] = false
- return false // no cycle found

}

Topological Sorting => Directed, Acyclic, Graph [DAG]

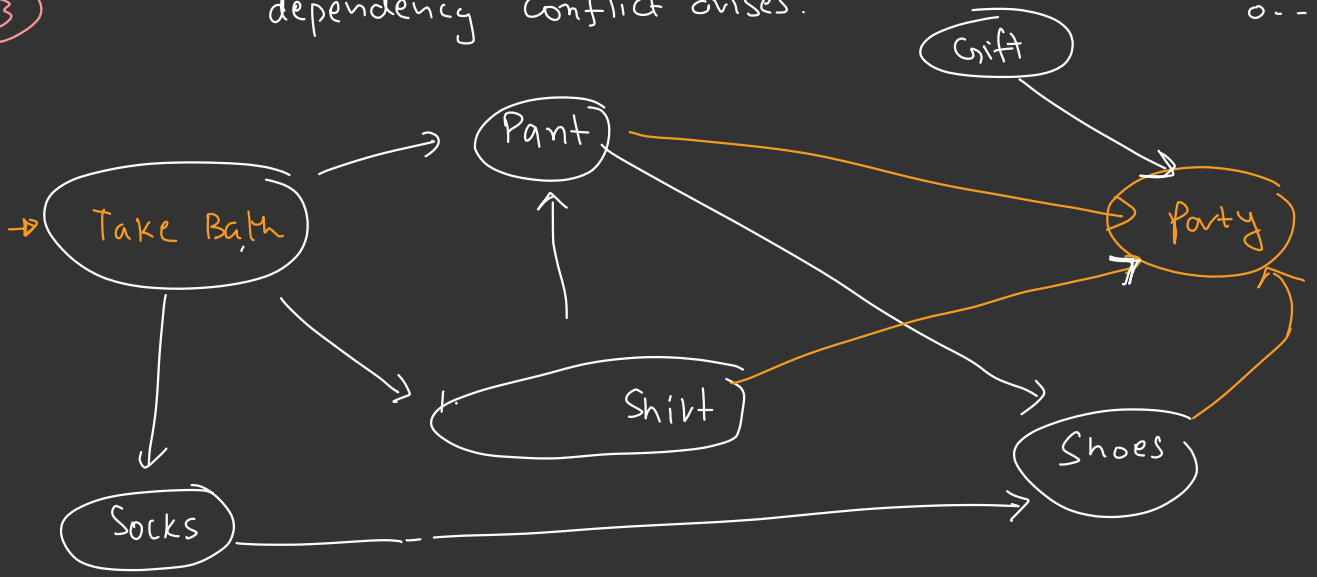
Given a list of Dependencies, find the correct order of tasks to be executed so that dependency conflict arises.

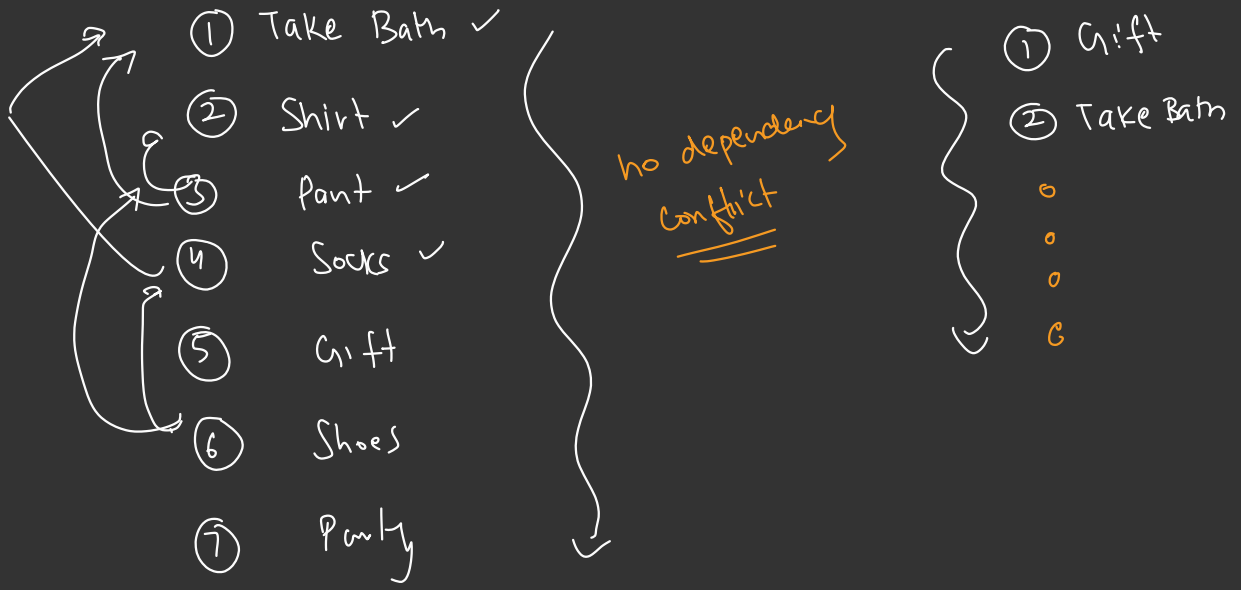
Nodes

0 to N-1

N Nodes

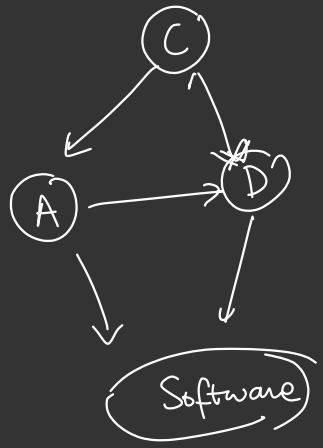
1 to N



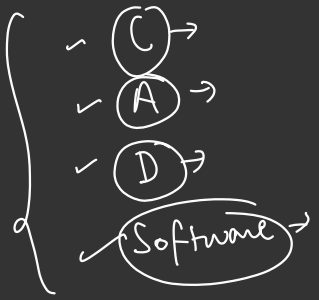

1 Take Bath ✓
2 Shirt ✓
3 Pant ✓
4 Socks ✓
5 Gift
6 Shoes
7 Party
No dependency conflict
1 Gift
2 Take Bath
C

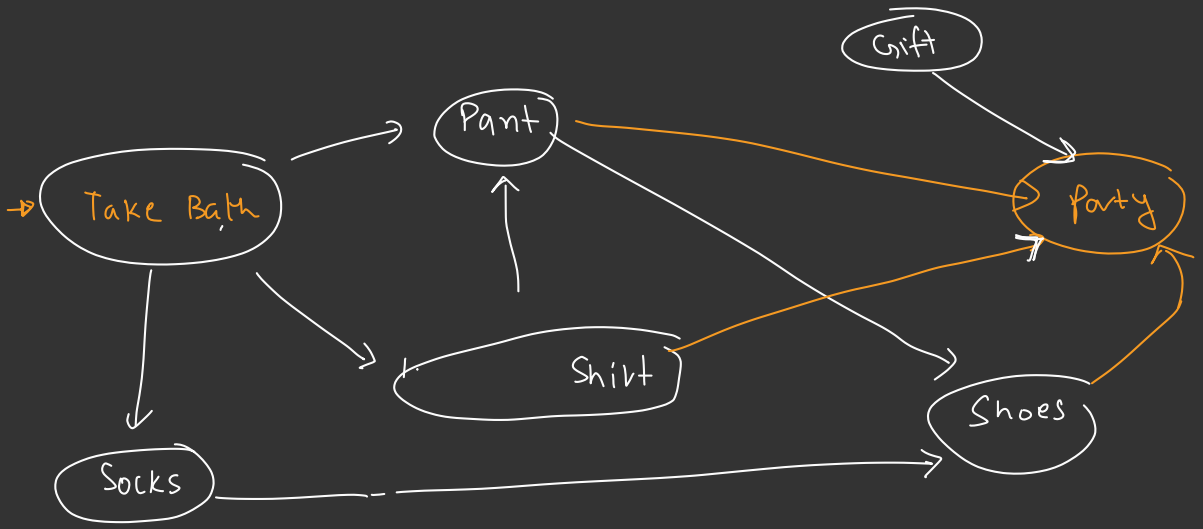

A
B
C
not possible for cyclic graphs.

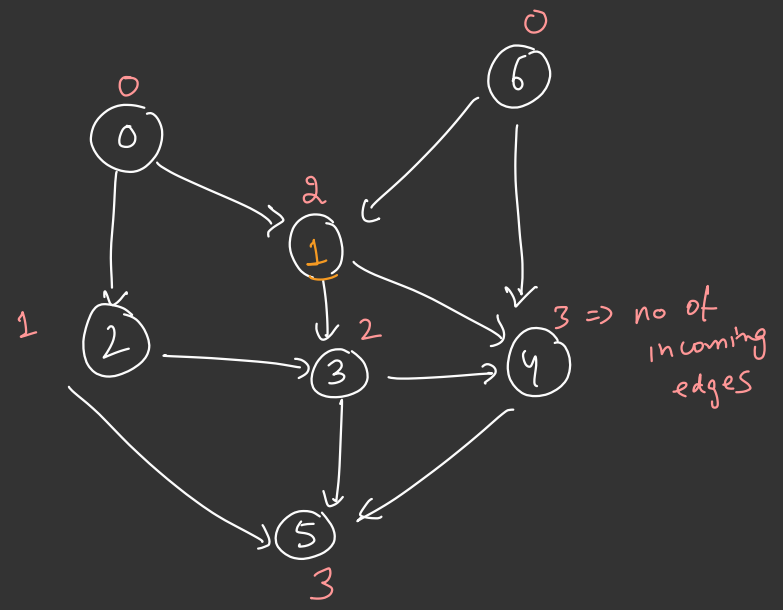
Real life Application



Building Process → compilation of a complex software




Gift
Pant
Take Bath
Party
Shirt
Shoes
Socks



adj list

0 → (1, 2)

1 → (3, 4)

2 → (3, 5)

6 → (1, 4)

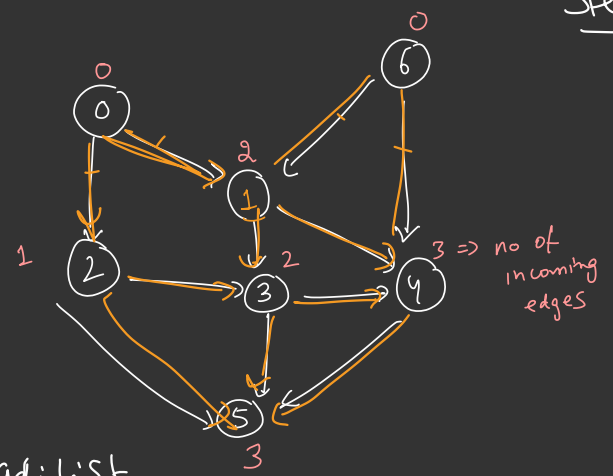
4 → (5)

5 → ( )

6 → (1, 4)

[DAG acyclic] ✓

ordering = _ _ _ _ _ _ _ _ _ (any valid ordering)



Step-1 Build Indegree Array

indegree =

0	2	1	2	3	3	0
0	1	2	3	4	5	6

```
for(i=0; i<N, i++){

    for(nbr . adjList[i]){

        indgree[nbr]++

    }

}
```

adjList

0 → (1, 2)

1 → (3, 4)

2 → (3, 5)

6 → (1, 4)

4 → (5)

5 → ( )

3 → (4, 5)

0	2	1	2	3	3	0	
0	1	2	3	4	5	6	

↑ ↑

Step-2 Identify Nodes with 0 Indegree

queue q

```
for (i=0; i<N; i++) {
    if (indgree[i] = 0){
        q.push(i);
    }
}
```

q

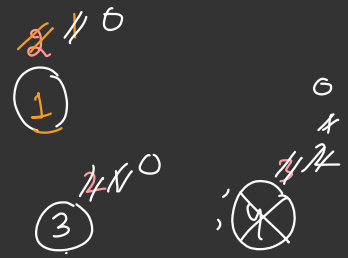
↑ doesn't have dep

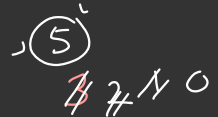
Step-3 Start doing task

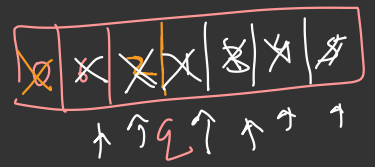
```
while ( !q.empty()) {
    task = q.poll()      print(task)
    for( nbr : adjList[task]) {
        indegree[nbr] --
        if( indegree[nbr] == 0){
            q.push(nbr)
        }
    }
}
```

output -> [0, 6, 2, 1, 3, 4, 5]

easy :)







10 Min break :)

10.47

Problem

https://www.hackerrank.com/challenges/journey-to-the-moon/problem

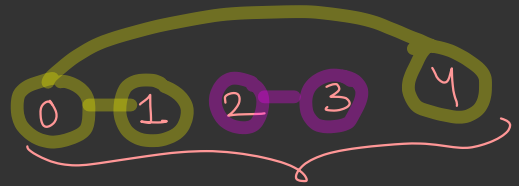
The member states of the UN are planning to send people to the moon. They want them to be from different countries. You will be given a list of pairs of astronaut ID's. Each pair is made of astronauts from the same country. Determine how many pairs of astronauts from different countries they can choose from.

No of V E

5 3 → No of pairs

Astroid
- 0 1 #1
- 2 3 #2
- 0 4 #3

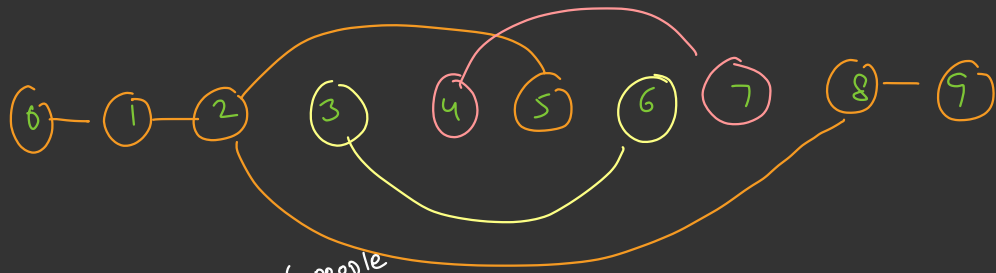
$O(V+E)$



5 Astronauts

$^3C_1 \times {}^2C_1 = 6$ ways

6 ways
- 0-2
- 0-3
- 1-4
- 1-3
- 4-2
- 4-3



Edge list

0 – 1
1 – 2
2 – 5
8 – 9
2 – 8
3 – 6
4 – 7
5 – 9

6 people

C#1

$^6C_2$

C# 2

$^2C_2$

C#3

$^2C_2$

→ DFS to find size of each connected component ⇒ denote.

Choose 2 astronauts / pair → diff country (K)

$$= \underline{C1}.C2 + \underline{C2}.C3 + \underline{C1}.\underline{C3}$$

$$= 6.2 + 2.2 + 6.2$$

= 28 ways.

Easier way

ans = Total ways to choose 2 astronauts − ways in which both astronauts belong to same country

formula

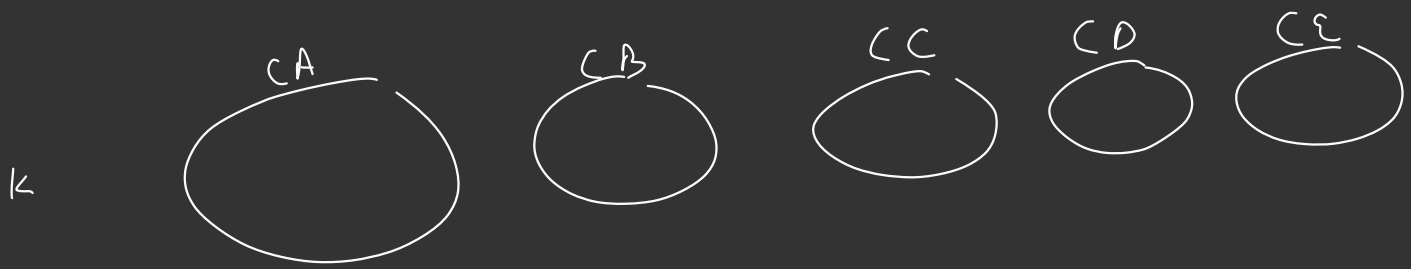
= $^{N=10}C_2$ − ($^{6}C_2$ + $^{2}C_2$ + $^{2}C_2$)

Loop K times; Size of each comp.

= $\frac{10 \cdot 9}{2}$ − ($\frac{6 \cdot 5}{2}$ + 1 + 1)

= 45 − (15 + 2)

= 28



K

I st — CA, CB — ... — every pair countries

CA, CC

CC,

$^{K}C_{2}$ Loop

II nd — every country

$${}^{N}C_{2} = \frac{N(N-1)}{2}$$

$$n \frac{(n-1)}{2}$$

$$\Rightarrow {}^{10}C_{2} = \frac{10 \cdot 9}{2} = \boxed{45}$$

$${}^{N}C_{R} = \frac{N!}{(N-R)!\, R!}$$